NOTES

1. Show your `:root` palette. Why are named variables better than repeating hex codes across the file?

The `:root` selector is used to define CSS variables that can be reused throughout the stylesheet.

Example:

```
:root {
    --primary: #2563eb;
    --secondary: #0f172a;
    --background: #f8fafc;
    --text: #1e293b;
    --white: #ffffff;
}
```

Instead of writing the same hex color repeatedly, we can use the variable name:

```
color: var(--text);
background-color: var(--background);
```

Named variables are better because:

- They make the CSS easier to read.
- The same color can be reused throughout the website.
- Changing a color only requires changing it in one place.
- They maintain consistency across the entire website.
- They make the stylesheet easier to maintain.

Therefore, CSS variables make the code more organized and maintainable.

---

2. Give one use each of hex, `rgb()` and `hsl()` from your CSS, and say where you used a semi-transparent colour and why.

CSS supports different ways of specifying colours.

Hex

Example:

background-color: #2563eb;

Hexadecimal colors are commonly used for fixed colours such as the primary color of buttons or headings.

RGB

Example:

color: rgb(255, 255, 255);

RGB represents a colour using Red, Green, and Blue values. It can be used for white text or other interface elements.

HSL

Example:

background-color: hsl(220, 80%, 50%);

HSL represents a colour using Hue, Saturation, and Lightness. It is useful when we want to easily adjust the brightness or saturation of a colour.

Semi-transparent colour

Example:

background-color: rgba(0, 0, 0, 0.5);

A semi-transparent colour was used for an overlay or shadow effect.

The `0.5` value represents 50% opacity. It allows some of the background image or content to remain visible while improving readability.

---

3. Why prefer `rem` over `px` for fonts and spacing? Show one `calc()` and one `clamp()` you used and what each does.

`rem` is preferred over `px` for fonts and spacing because `rem` is relative to the root element's font size.

For example:

font-size: 1.5rem;
padding: 2rem;

Using `rem` helps create a more scalable and accessible design. If the root font size changes, the related sizes can adjust accordingly.

`calc()`

Example:

width: calc(100% - 2rem);

`calc()` performs calculations between different CSS units. In this example, the element takes the full available width minus `2rem`.

`clamp()`

Example:

font-size: clamp(1.5rem, 5vw, 3rem);

`clamp()` sets a minimum, flexible, and maximum value.

Here:

- `1.5rem` = minimum size
- `5vw` = flexible size based on viewport width
- `3rem` = maximum size

This allows the heading to resize smoothly according to the screen size without using a media query.

---

4. A box has `width: 200px; padding: 20px; border: 5px`. What is its actual rendered width with `content-box`, and with `border-box`? Which did you use and why?

With `content-box`

In the default `content-box` model, the declared width applies only to the content.

Content = 200px
Left + Right Padding = 20px + 20px
Left + Right Border = 5px + 5px

Therefore:

Actual width = 200 + 20 + 20 + 5 + 5
= 250px

So, the rendered width is **250px**.

With `border-box`

With `border-box`, the declared width includes the content, padding, and border.

Total width = 200px

So, the rendered width is **200px**.

A common CSS rule is:

```
* {
    box-sizing: border-box;
}
```

I used `border-box` because it makes sizing easier and more predictable. The declared width already includes padding and borders, so elements are less likely to overflow unexpectedly.

---

5. Name the four layers of the box model, from the inside out.

The four layers of the CSS box model, from inside to outside, are:

1. **Content** – The actual text, image, or other content.
2. **Padding** – Space between the content and the border.
3. **Border** – The line surrounding the padding and content.
4. **Margin** – Space outside the border that separates the element from other elements.

The order is:

Content
↓
Padding
↓
Border
↓
Margin

The CSS box model helps us understand how the size and spacing of an element are calculated.

---

6. What does `repeat(auto-fit, minmax(15rem, 1fr))` do, and why does it need no media query?

The following CSS creates a responsive grid:

grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));

Here:

- `repeat()` repeats the column pattern.
- `auto-fit` automatically fits as many columns as possible.
- `minmax(15rem, 1fr)` gives each column a minimum width of `15rem` and allows it to grow up to `1fr`.

The number of columns automatically changes according to the available screen width.

For example:

Large screen → 4 columns
Medium screen → 2 or 3 columns
Small screen → 1 column

It does not require a media query because CSS Grid automatically calculates how many columns can fit in the available space.

---

7. Mobile-first: why write the phone styles as the base and add layout with `min-width` queries? And why is the viewport `<meta>` tag essential?

Mobile-first design means writing the basic styles for mobile devices first.

Example:

```
.container {
   display: block;
}
```

Then larger-screen layouts are added using `min-width` media queries:

```
@media (min-width: 768px) {
   .container {
      display: grid;
   }
}
```

Mobile-first design is useful because:

- Mobile screens have limited space.
- The basic design remains simple.
- Larger-screen features can be added progressively.
- It improves responsive design and maintainability.

The viewport meta tag is:

<meta name="viewport" content="width=device-width, initial-scale=1.0">

It tells the browser to use the actual device width and set the initial zoom correctly.

Without the viewport tag, a website may appear very small on mobile devices because the browser may render it using a wider virtual layout.

---

8. Paste your `grid-template-areas` map. Which regions span more than one column, and how did you make them?

Example of the `grid-template-areas` map:

```
grid-template-areas:
   "header header"
   "sidebar main"
   "footer footer";
```

The layout can be represented as:

```
+------------------------+
|        HEADER          |
+-------------+----------+
|  SIDEBAR    |   MAIN   |
+-------------+----------+
|        FOOTER          |
+------------------------+
```

The `header` and `footer` regions span more than one column.

They are made to span two columns by repeating their area names:

"header header"

and

"footer footer"

The areas can then be assigned to elements:

```
header {
    grid-area: header;
}

aside {
    grid-area: sidebar;
}

main {
    grid-area: main;
}

footer {
    grid-area: footer;
}
```

---

9. Name one responsive technique for images and one for type that need no media query.

Responsive image

One simple technique is:

```
img {
    max-width: 100%;
    height: auto;
}
```

This makes the image fit inside its container and prevents it from overflowing on smaller screens.

Responsive typography

For responsive text, `clamp()` can be used:

```
h1 {
   font-size: clamp(1.8rem, 5vw, 3rem);
}
```

The font size changes according to the viewport width while staying between the minimum and maximum values.

Therefore, no media query is required.

---

10. Prepare a GitHub repo for `event-board` and paste your final `index.html` and final `style.css` — the outcome of all the assignments.

The final project should be uploaded to a GitHub repository named:

event-board

A suitable repository structure is:

```
event-board/
│
├── index.html
├── style.css
├── README.md
└── images/
```

Final `index.html`

Paste the final version of the HTML file used in the completed project here:

<!-- Paste your final index.html code here -->

Final `style.css`

Paste the final version of the CSS file used in the completed project here:

/* Paste your final style.css code here */

GitHub Repository

GitHub Repository:
https://github.com/chandrakant-123/Campus_Event_Board_IIITV

.GitHub Pages

If GitHub Pages is enabled:

GitHub Pages:
https://YOUR-USERNAME.github.io/event-board/

If Pages is not enabled, write:

GitHub Pages:
Not deployed

README

The repository contains a `README.md` file with:

- Project name
- Project description
- Features
- Technologies used
- How to run the project
- Live demo link, if available

Commit History

The repository should have a readable commit history, for example:

Initial project setup
Add event board HTML structure
Add CSS styling

Add CSS Grid layout
Add responsive design
Improve mobile layout
Add README
Final project update

Therefore, the final GitHub repository contains the completed `index.html`, `style.css`, `README.md`, project assets, and a readable commit history.

LAB ASSIGNMENT 6 - ANSWERS

Campus Event Board | Section A: Language basics & logic | Section B: DOM, events, fetch & validation

1. let vs const - when do you use each, and why avoid var?

Use **const** when a variable will not be assigned a new value. Use **let** when its value must change. In this project, the events variable uses let because it starts as an empty array and receives the data after fetch(). Variables such as menuButton and eventGrid use const because those references do not need reassignment. I avoid var because it is function-scoped and can be redeclared, which can make code harder to follow. let and const use block scope.

2. List the data types you used and give one value of each.

The events.json objects use a **string** such as "HACER 2025 Conference" for title, a **number** such as 0 for seats, and a **boolean** such as false for online. The script also uses strings for messages and dates, numbers for array lengths, and booleans for checks such as whether an event is online.

3. Show one event object. Why is an array of objects a good model?

Example object:

```
{
  "title": "HACER 2025 Conference",
  "date": "19-20 Dec. 2025",
  "seats": 0,
  "online": false
}
```

An array of objects keeps the details of each event together in one record. The page can loop through the array to create cards, search the records, and show availability. events.json is the single source of truth for the event cards.

4. Show the control-flow rule from A4. What labels did it produce?

```
if (eventItem.seats === 0) return 'Full';
else if (eventItem.seats < 5) return 'Few Left';
else return 'Open';
```

With the sample seat values in events.json, HACER 2025 Conference is Full; Rashtriya Karmayogi Workshop is Few Left; 7th Inter IIIT Sports Meet is Open; Reduce E-Waste Recognition is Few Left; and 3-Day AI Accelerate Boot Camp is Open. Seat counts are sample values for this programming exercise.

5. Show one function (A6) and one arrow function with an array method (A7). What does each return?

```
function formatEvent(eventItem) {
  const mode = eventItem.online ? 'Online' : 'On campus';
  return `${eventItem.date} | ${eventItem.seats} seats left | ${mode}`;
}

const filterEvents = (eventList, searchText) => {
  const searchValue = searchText.toLowerCase().trim();
  return eventList.filter((eventItem) => {
    return eventItem.title.toLowerCase().includes(searchValue) ||
      eventItem.category.toLowerCase().includes(searchValue);
  });
};
```

formatEvent(eventItem) returns one readable string containing the date, seats left, and mode. filterEvents(eventList, searchText) returns a new array containing only the events whose title or category matches the search text.

A8 - Scope: eventCount is declared with const inside renderEvents(). It belongs to that function scope, so code outside renderEvents() cannot access it. Keeping it local avoids creating a global variable.

LAB ASSIGNMENT 6 - ANSWERS (CONTINUED)

6. Which elements did you select, and with which methods? Why prefer querySelector?

The script selects the hamburger button, navigation, event grid, event status, registration form, and form message with document.querySelector(). It selects all navigation links with document.querySelectorAll(). It selects event-search, name, and other elements by ID with document.getElementById(). Each selected element reference is stored in a const variable. querySelector is useful because it accepts CSS selectors such as a class, ID, or attribute selector and returns the first matching element.

7. Explain event bubbling in one or two lines, using event.target from B5.

When an RSVP button is clicked, the click event travels from the button to its parent elements, including the event grid. This is event bubbling. The event.target property identifies the exact RSVP button that was clicked, so one listener on the event grid can handle all the buttons.

8. Describe the three states your fetch handles, and how the user sees each.

Loading: the status area displays "Loading events..." while the browser requests events.json. **Success:** after the JSON is received, the script displays the event cards and a message such as "Events loaded successfully. 5 event(s) displayed." **Error:** if the request fails or the response is not usable, the event grid is cleared and a friendly message asks the user to check events.json and refresh the page. The catch block also writes the error to the Console.

9. Show the built-in validation attributes and custom rule. Why must the server validate again?

The name, email, mobile number, and password inputs use required. The email input uses type="email". The mobile input uses pattern="[0-9]{10}" for a 10-digit number. The custom rule uses nameInput.setCustomValidity() to require at least two non-space characters. On submit, the script calls preventDefault(), checks registrationForm.checkValidity(), and uses reportValidity() to show browser feedback. The server must validate again because browser checks can be bypassed or changed by a user; only server-side validation can protect data at the point where it is received.

10. Give the GitHub repo link; confirm README and commit history; paste final script.js.

The Lab 5 notes already record this repository address: https://github.com/chandrakant-123/Campus_Event_Board_IIITV. The updated Lab 6 files in this package have **not** been pushed to GitHub as part of this work. README.md is included in the local project folder. The repository commit history still needs to be updated after these files are uploaded. GitHub Pages deployment has not been verified.

The complete final script.js is included in Appendix A on the following pages. The five sample event records and seat values are kept in events.json, not repeated as hard-coded HTML cards.

APPENDIX A - FINAL script.js

Complete JavaScript file used by index.html. Code comments identify the Lab 6 requirements.

```
// Lab Assignment 6 - Campus Event Board

// B1: Select the HTML elements needed by the JavaScript.
const menuButton = document.querySelector('.menu-toggle');
const navigation = document.querySelector('.nav-links');
const navigationLinks = document.querySelectorAll('.nav-links a');
const eventGrid = document.querySelector('.event-grid');
const eventSearch = document.getElementById('event-search');
const eventStatus = document.querySelector('#event-status');
const registrationForm = document.querySelector('.registration form');
const nameInput = document.getElementById('name');
const formMessage = document.querySelector('#form-message');

// A3 + B7: The events array is loaded from events.json and is the single source of event data.
// A2: let is used because this array receives its value after fetch().
let events = [];

// A4: if / else decides the availability label from the number of seats.
function getSeatStatus(eventItem) {
    if (eventItem.seats === 0) {
        return 'Full';
    } else if (eventItem.seats < 5) {
        return 'Few Left';
    } else {
        return 'Open';
    }
}

// A6: This function accepts one event and returns a readable line.
function formatEvent(eventItem) {
    const mode = eventItem.online ? 'Online' : 'On campus';
    return `${eventItem.date} | ${eventItem.seats} seats left | ${mode}`;
}

// A7: Arrow function + filter() searches the event title and category.
const filterEvents = (eventList, searchText) => {
    const searchValue = searchText.toLowerCase().trim();

    return eventList.filter((eventItem) => {
        return eventItem.title.toLowerCase().includes(searchValue) ||
            eventItem.category.toLowerCase().includes(searchValue);
    });
};

// B2 + B3 + A5: Create the cards from the data using createElement() and a for...of loop.
function renderEvents(eventList) {
    // A8: eventCount is local to this function and cannot be used outside it.
    const eventCount = eventList.length;
    eventGrid.textContent = '';

    if (eventCount === 0) {
        eventStatus.textContent = 'No events match your search.';
        return;
    }
```

APPENDIX A - FINAL script.js (CONTINUED)

```
// A5: Visit each event in the list once while building the visible cards.
for (const eventItem of eventList) {
    const card = document.createElement('article');
    card.classList.add('event-card');
    card.classList.toggle('event-card--online', eventItem.online);
    card.classList.toggle('full', eventItem.seats === 0);
    card.setAttribute('aria-label', eventItem.title);

    const image = document.createElement('img');
    image.classList.add('event-card__image');
    image.setAttribute('src', eventItem.image);
    image.setAttribute('alt', eventItem.imageAlt);
    image.setAttribute('loading', 'lazy');

    const body = document.createElement('div');
    body.classList.add('event-card__body');

    const category = document.createElement('p');
    category.classList.add('event-card__tag');
    category.textContent = eventItem.category;

    const title = document.createElement('h3');
    title.textContent = eventItem.title;

    const description = document.createElement('p');
    description.classList.add('event-card__text');
    description.textContent = eventItem.description;

    const details = document.createElement('p');
    details.classList.add('event-card__details');
    details.textContent = formatEvent(eventItem); // A6: Use the function's returned string.

    const seatStatus = document.createElement('p');
    const statusText = getSeatStatus(eventItem);
    seatStatus.classList.add('event-card__status');
    seatStatus.classList.add(`status-${statusText.toLowerCase().replace(' ', '-')}`);
    seatStatus.textContent = statusText;
    seatStatus.style.fontWeight = '700'; // B3: Use the style property to highlight availability.

    // B5: This button lets the click handler demonstrate event bubbling.
    const rsvpButton = document.createElement('button');
    rsvpButton.classList.add('rsvp-button');
    rsvpButton.setAttribute('type', 'button');
    rsvpButton.setAttribute('aria-label', `RSVP interest for ${eventItem.title}`);
    rsvpButton.dataset.eventTitle = eventItem.title;
    rsvpButton.textContent = eventItem.seats === 0 ? 'Full' : 'RSVP Interest';
    rsvpButton.disabled = eventItem.seats === 0;

    body.appendChild(category);
    body.appendChild(title);
    body.appendChild(description);
    body.appendChild(details);
    body.appendChild(seatStatus);
    body.appendChild(rsvpButton);
    card.appendChild(image);
```

APPENDIX A - FINAL script.js (CONTINUED)

```
        card.appendChild(body);
        eventGrid.appendChild(card);
    }

    eventStatus.textContent = `${eventCount} event(s) displayed.`;
}

// B7: Fetch the event data with async/await and handle Loading, Success and Error states.
async function loadEvents() {
    eventStatus.textContent = 'Loading events...';

    try {
        const response = await fetch('events.json');

        if (!response.ok) {
            throw new Error('Could not load events.json.');
        }

        const eventData = await response.json();

        if (!Array.isArray(eventData)) {
            throw new Error('The event data must be an array.');
        }

        events = eventData;
        console.log('Events loaded successfully:', events); // B9: Inspect event data in DevTools Console.
        renderEvents(events);
        eventStatus.textContent = `Events loaded successfully. ${events.length} event(s) displayed.`;
    } catch (error) {
        console.error('Event loading error:', error); // B9: Inspect fetch errors in DevTools Console.
        eventGrid.textContent = '';
        eventStatus.textContent = 'Sorry, events could not be loaded. Please check events.json and refresh the page.';
    }
}

// B4: The hamburger button opens and closes the existing navigation list.
menuButton.addEventListener('click', () => {
    const menuIsOpen = navigation.classList.toggle('is-open');
    menuButton.setAttribute('aria-expanded', menuIsOpen);
    menuButton.setAttribute('aria-label', menuIsOpen ? 'Close navigation menu' : 'Open navigation menu');
});

// B4: Close the mobile menu after a navigation link is selected.
for (const link of navigationLinks) {
    link.addEventListener('click', () => {
        navigation.classList.remove('is-open');
        menuButton.setAttribute('aria-expanded', 'false');
        menuButton.setAttribute('aria-label', 'Open navigation menu');
    });
}

// B5: The click bubbles from an RSVP button up to the eventGrid container.
eventGrid.addEventListener('click', (event) => {
    // event.target identifies the exact element that was clicked.
    if (event.target.classList.contains('rsvp-button')) {
```

APPENDIX A - FINAL script.js (CONTINUED)

```
        const eventTitle = event.target.dataset.eventTitle;
        eventStatus.textContent = `Interest noted for: ${eventTitle}. (Demo only)`;
        console.log('RSVP button clicked:', eventTitle);
    }
});

// B6: Filter the events whenever the user types in the search box.
eventSearch.addEventListener('input', () => {
    const matchingEvents = filterEvents(events, eventSearch.value);
    renderEvents(matchingEvents);
});

// B6: Pressing Escape clears the search and displays all events again.
eventSearch.addEventListener('keydown', (event) => {
    if (event.key === 'Escape') {
        eventSearch.value = '';
        renderEvents(events);
    }
});

// B8: Custom validation rule - the name must contain at least 2 non-space characters.
function checkName() {
    if (nameInput.value.trim().length < 2) {
        nameInput.setCustomValidity('Please enter a name with at least 2 characters.');
    } else {
        nameInput.setCustomValidity('');
    }
}

nameInput.addEventListener('input', checkName);

// B8: Stop the demo form submission, check all fields, and display the result.
registrationForm.addEventListener('submit', (event) => {
    event.preventDefault();
    checkName();

    if (!registrationForm.checkValidity()) {
        formMessage.textContent = 'Please correct the highlighted fields and try again.';
        registrationForm.reportValidity();
        console.log('Form validation: invalid'); // B9: Check the handler in DevTools Console.
        return;
    }

    formMessage.textContent = 'Your details passed the form validation (demo only).';
    console.log('Form validation: successful');
});

// B8: Clear the custom validation message when the user resets the form.
registrationForm.addEventListener('reset', () => {
    nameInput.setCustomValidity('');
    formMessage.textContent = '';
});

// Start loading the event data after the page elements and handlers are ready.
loadEvents();
```